BioOpenClaw：多实例 AI 生物信息科学家群框架设计文档

> 版本 v0.1 | 2026-03-13 | 状态：草稿

一、框架概述

BioOpenClaw 是一个基于多 OpenClaw 实例的 AI 生物信息科学家群系统。其核心理念是：**每一个 OpenClaw 实例是一个具有独立身份、记忆和专业能力边界的 AI 科学家角色**，多个实例在指挥层的协调下并行工作，共同完成复杂的生物信息研究任务。

与"单一 OpenClaw + Skills 插件"的方案不同，本框架将不同性质的科研工作分配给专精实例，从进程级别实现真正的职责隔离与并行执行。

二、系统架构

```
┌──────────────
```

职责定位：数据的全生命周期管理——从外部来源获取原始数据，到清洗、归一化、格式转换，输出可用于下游分析的标准化数据集。

核心能力：

- 从 NCBI、GEO、UniProt、PDB、TCGA、CellxGene Census 等数据库自动检索和下载数据
- 单细胞 RNA-seq 数据预处理（质控、归一化、批次校正）
- 多组学数据整合（CITE-seq、Multiome、Spatial Transcriptomics）
- 数据格式标准化（输出统一为 AnnData .h5ad 格式）
- 数据集版本管理和 Checksum 校验

主要工具栈：Biopython / GEOparse / bioservices / Scanpy / scvi-tools / AnnData / Muon

SOUL.md 行为边界（草稿）：

> 你是一名专精于生物数据工程的 AI 科学家。
> 你的工作范围严格限定于数据获取、处理和归一化。
> 你不做模型训练，不写研究假设，不生成图表。
> 当收到超出范围的任务时，拒绝并通知指挥层重新分配。

部署节点类型：网络带宽优先（高 IO 节点）/ CPU 计算实例

3.2 模型 OpenClaw（Model Agent）

职责定位：跟踪生物信息 AI 领域最新模型，负责模型的发现、下载、部署、微调和推理服务维护。

核心能力：

- 持续监控 arXiv/bioRxiv/HuggingFace Hub 的最新生物基础模型（ESM2、scGPT、Geneformer、Evo2、OpenFold3 等）
- 自动化模型下载、版本管理、环境配置
- 基于 LoRA/QLoRA/PEFT 的模型微调流水线（支持 ESM2、scGPT、Geneformer）
- 模型推理 API 封装和服务化（FastAPI + Triton）
- 超参数搜索（Optuna）与训练监控（MLflow）

主要工具栈：HuggingFace Hub / PEFT / DeepSpeed / BioNeMo / MLflow / Optuna / SageMaker

SOUL.md 行为边界（草稿）：

> 你是一名专精于 AI 模型工程的科学家。
> 你负责模型的发现、部署和微调，不做原始数据处理，不做研究设计。
> 微调任务必须接收来自数据 Agent 输出的标准化数据，不直接访问原始数据库。

部署节点类型：GPU 实例（A100/H100，用于微调）/ 普通计算实例（用于模型监控）

3.3 研究 OpenClaw（Research Agent）

职责定位：科研的"大脑"——文献挖掘、假设生成、实验方案设计、结果分析与比对、报告撰写。

核心能力：

- 自动文献检索与整理（PubMed / Semantic Scholar / bioRxiv API）
- 生物医学 NER 与知识图谱构建（PubTator 3.0 / PyKEEN / Neo4j）
- RAG 系统（SPECTER2 嵌入 + Qdrant 向量库）检索支持
- 研究假设生成与创新性评估
- 实验方案优化（Ax Platform / BoTorch 贝叶斯优化）
- 结果统计检验与多模型对比（Pingouin / scIB / TDC 基准）
- 科研图表生成（Scanpy / matplotlib / py3Dmol / pyGenomeTracks）
- 结构化报告和论文草稿输出

主要工具栈：Semantic Scholar API / PubTator / SPECTER2 / Qdrant / Neo4j / Ax / Pingouin / Scanpy / MLflow

SOUL.md 行为边界（草稿）：

> 你是一名专精于生物信息科研分析的 AI 科学家。
> 你基于数据 Agent 提供的数据集和模型 Agent 提供的推理结果开展研究。
> 你负责科学判断和报告撰写，不直接操作数据库，不部署模型。
> 关键结论（假设选择、实验设计）需标记为"待人工确认"后再通知指挥层推进。

部署节点类型：高并发 LLM API 节点 / 通用计算实例

3.4 指挥层（Commander Layer）

设计哲学：指挥层是整个系统的调度核心，但**不应该是另一个 OpenClaw LLM 实例**。

指挥层分为两个子层：

4a. 人工指挥（Human Commander）

- 定义研究目标和优先级
- 在关键节点审批（假设选择、实验方案确认、报告发布）
- 处理 Agent 之间的冲突和异常
- 通过结构化 Prompt 下达指令（推荐使用标准化的 Task Spec JSON）

4b. 自动调度脚本（Auto Dispatcher）

- 工具：LangGraph Supervisor + Temporal 持久化
- 职责：任务分解（DAG 拆解）、依赖管理（数据就绪才触发研究 Agent）、状态追踪、失败重试
- **强调**：调度逻辑用代码实现（确定性、可审计），不依赖 LLM 做路由判断

任务状态机（简化）：

```
PENDING → ASSIGNED → RUNNING → COMPLETED
                       ↓
                  FAILED → RETRY（最多 3 次）→ ESCALATE（人工介入）
```

四、Agent 间通信协议

通信场景	协议	说明
指挥层 → 子 Agent 任务分配	REST / MCP	同步调用，等待确认收到
子 Agent → 指挥层 结果返回	Redis Pub/Sub	异步通知，结果写入共享存储
子 Agent 间数据传递	S3 + 事件通知	大文件通过对象存储共享，不直接传输
长任务状态同步	Temporal Workflow	持久化执行，支持断点恢复
紧急中断	Redis 消息	指挥层可随时发送 ABORT 信号

数据共享约定：所有跨 Agent 的数据交换以 AnnData `.h5ad`（单细胞数据）或 JSON（元数据和任务指令）为标准格式，存储在共享 S3 存储桶，Agent 不直接互相传输文件。

五、典型工作流示例

以"寻找 BRCA1 突变相关的细胞类型特异性调控网络"为例：

```
人工指挥下达目标
    │
    ▼
指挥层（Auto Dispatcher）分解任务：
  Task-1: [数据 OpenClaw] 从 TCGA + GEO 下载 BRCA1 突变相关单细胞数据
  Task-2: [研究 OpenClaw] 检索近 5 年 BRCA1 调控网络相关文献（与 Task-1 并行）
    │
    ▼（Task-1 完成后触发）
  Task-3: [模型 OpenClaw] 用 Task-1 数据微调 Geneformer，强化 BRCA1 扰动预测能力
    │
    ▼（Task-3 完成后触发）
  Task-4: [研究 OpenClaw] 基于微调模型推理结果 + 文献知识图谱，生成研究假设
    │
    ▼
人工确认检查点：审批假设方案
    │（批准）
    ▼
  Task-5: [研究 OpenClaw] 执行统计检验，生成图表和研究报告草稿
    │
    ▼
人工审核报告 → 完成
```

六、与"单 OpenClaw + Agent + Skill"方案的对比

维度	单实例 + Skills	多 OpenClaw 实例（本框架）
上下文质量	所有工具混合，上下文噪声大	每个实例只有本领域的干净上下文
并行能力	Sub-Agent 有限并行，受单进程制约	进程级真并行，数据处理与文献挖掘可同时进行
故障隔离	一个 Skill 崩溃可能影响整体	单个实例崩溃不影响其他实例
资源分配	统一分配，无法差异化	数据 Agent 高带宽节点，模型 Agent 独占 GPU
行为稳定性	Prompt 控制，易被上下文污染	SOUL.md 身份固化，边界清晰
LLM 成本优化	全部任务用同一模型	不同 Agent 按任务复杂度选模型（Haiku vs Sonnet）
开发复杂度	✅ 更低，单点维护	❌ 更高，需要设计跨进程通信和调度
调试难度	✅ 更容易	❌ 分布式调试难度更大
扩展性	受单进程限制	任一 Agent 可独立水平扩展

结论：对于简单任务或快速原型，单实例方案更轻量；对于长期运行的、需要并行处理的复杂生物信息研究流水线，多实例方案的上下文隔离和并行能力优势是决定性的。

七、架构评注（Claude Comments）

✅ 框架设计的核心正确性

这个框架的**根本直觉是正确的**：生物信息科研工作本身就是高度并行的——数据工程师、计算生物学家、AI 工程师在真实科研组里是不同的人，同时工作。将这种分工映射到多 Agent 实例，不是为了"炫技"，而是真实反映了科研工作的分工结构。CellForge（耶鲁大学 Gerstein Lab，arXiv:2508.02276）已经验证了类似的多 Agent 分工可以在单细胞建模上超越单模型基线 49%，说明这个方向是有实验依据的。

⚠️ 最需要警惕的工程风险

风险 1：指挥层的复杂度被低估

三个 Agent 之间的依赖关系在任务量增加后会迅速变成一个复杂的 DAG。建议从第一天就用 Temporal 或 Prefect 做工作流持久化，而不是自己维护状态机——分布式状态管理是经典的工程陷阱。

风险 2：OpenClaw 的 Node.js 运行时与 Python 生物信息工具的摩擦

OpenClaw 核心是 Node.js/TypeScript，而 Scanpy、ESM2、RDKit 全是 Python 生态。这个 gap 不是不可以跨越，但在调试时会消耗大量时间。建议在 Phase 1 验证阶段就把这个桥接问题解决，不要留到后期。最稳健的方案是：将每个 Python Agent 封装为独立的 MCP Server，OpenClaw 通过 MCP 协议调用，彻底隔离运行时。

风险 3：LLM 在研究 Agent 中的幻觉风险

研究 OpenClaw 产出的假设和结论会直接影响科研方向。必须在每一个输出节点设计验证机制：文献引用必须可追溯到原始 API 返回结果，统计结论必须附带原始数据和检验代码，禁止 Agent 凭记忆给出数值。建议每个关键科学声明都附带"来源 + 可复现代码"，否则宁可不输出。

💡 建议的一个架构调整

当前设计中"指挥 OpenClaw"是可选的——你也提到可以是人工指挥。**建议默认走"人工 + 轻量调度脚本"的混合模式**，而不是再加一个 LLM 指挥 Agent。原因是：任务路由和依赖管理是确定性问题（适合代码），而不是推理问题（适合 LLM）。引入 LLM 做调度只会增加不确定性和成本。LLM 的价值在各专精 Agent 内部的"理解和生成"，指挥层的调度逻辑用 Python DAG 更稳定可靠。

人工介入的最优节点不是全流程监控，而是两个卡点：（1）研究假设生成后、正式实验前；（2）分析结论形成后、报告发布前。其余流程让自动调度跑，既保持人工控制权，又不成为系统瓶颈。

🔭 值得关注的扩展方向

未来可以在此框架上扩展第四个专精实例——**AIDD OpenClaw（药物发现 Agent）**，集成 REINVENT4 + DiffDock + ADMET-AI，承接"从靶点到候选分子"的全链路任务。当前三实例架构为这

个扩展预留了良好的接口。

八、技术选型快速参考

层次	组件	推荐方案
Agent 框架	OpenClaw 实例	github.com/openclaw/openclaw
后端编排	任务调度	LangGraph + Temporal
LLM 路由	多模型管理	LiteLLM Proxy
消息通信	Agent 间异步	Redis Pub/Sub / AWS SQS
数据共享	大文件传输	AWS S3 / GCS
向量检索	RAG 支持	Qdrant
知识图谱	文献关系	Neo4j
工作流持久化	断点恢复	Temporal
实验追踪	模型训练	MLflow
基础设施即代码	云端部署	Pulumi（Python）

文档持续更新中。下一步：Phase 1 技术验证计划（MCP 桥接 + 最小调度链路）。